The AI Operating System

A complete beginner's guide to running real work on Claude Code — from "I've never opened a terminal" to a system that remembers your business, runs your workflows, and grades its own output before you send it.

PRACTITIONER GUIDE · V2.0 · AUGUST 2026 · WRITTEN FOR SOMEONE STARTING FROM ZERO

> Read this front to back once, then use it as a reference. Part 0 assumes you know nothing — not the terminal, not git, not what a "model" is. Parts 1–4 are the system itself, with every file you need written out in full so you can copy and paste. Parts 5–9 are how it's used for real client work. Part 10 is what to do when it breaks.

How to use this guide

Don't try to build everything at once. Read Part 0 and Part 1 today. Do the Day-1 exercise in Part 11. Then come back for one more part per day. The system compounds — the value comes from a week of small correct steps, not one heroic weekend.

PART 0 — GROUND FLOOR (START HERE)

0.1 What Claude Code actually is
0.2 The terminal, in 5 minutes
0.3 Installing it
0.4 Your first 20 minutes
0.5 The permission model
0.6 Glossary of every term
0.7 Where every file lives

PART 1 — THE IDEA

1.1 Chat box vs operating system
1.2 The four layers
1.3 Rules vs skills vs memory

PART 2 — BUILDING THE SYSTEM

2.1 The rules file (CLAUDE.md)
2.2 Memory — the brain
2.3 Skills — frozen procedures
2.4 Slash commands

2.5 Subagents
2.6 Parallelism
2.7 Hooks — automation
2.8 MCP — connecting tools

PART 3 — RUNNING IT DAILY

3.1 The daily loop
3.2 Working modes
3.3 Context hygiene

PART 4 — QUALITY

4.1 Quality gates
4.2 The critic loop
4.3 Green is not good

PART 5–9 — REAL WORK

5 Web & client delivery
6 Video production
7 Content & distribution
8 Automation (n8n / VPS)
9 Paid ads discipline

PART 10–12 — HARD RULES, FIXES, START

10.1 Hard rules (scar tissue)
10.2 Machine & cost discipline
10.3 Security hygiene
11 Troubleshooting
12 Build your own in 7 days
13 One-page cheat sheet

PART 0

Ground Floor

Skip nothing here if you're new. Every later part assumes these seven sections.

0.1 — What Claude Code actually is

Claude Code is an AI assistant that runs **in your terminal** and can **use your computer**. That second half is the whole point.

A normal AI chat can only produce text. You then do the work: copy the code, paste it into a file, save it, run it, read the error, paste the error back. You are the hands.

Claude Code has its own hands. It can:

- Read and write files on your disk
- Run commands (install packages, run tests, start servers, encode video)
- Use git — commit, branch, open pull requests
- Browse the web, call APIs, drive a headless browser
- Launch copies of itself to work on several things at the same time

So instead of "here's the code for that fix," you get: it reads the file, makes the change, runs the test, sees the test fail, reads the error, fixes it again, re-runs, and tells you it passes. You review the result rather than performing every step.

> **What it is not**
>
> It is not magic and it is not autonomous by default. It asks permission before doing anything meaningful, it makes mistakes, and it will confidently do the wrong thing if your instructions are ambiguous. The system in this guide exists precisely to constrain those failure modes.

Where it runs

Same tool, several front doors: the terminal (the original and most capable), a desktop app for Mac and Windows, a web app, and extensions for VS Code and JetBrains editors. This guide assumes the terminal, because everything else is a wrapper around it and the terminal is where the full system works.

0.2 — The terminal, in 5 minutes

The terminal (also called the command line, shell, console, or on Windows, PowerShell / Command Prompt) is a text window where you type commands instead of clicking. That's the entire concept.

Opening it

SYSTEM	HOW
Windows	Press `Win`, type `PowerShell`, press Enter. (Or install Windows Terminal from the Microsoft Store — nicer, same thing.)
macOS	Press `Cmd`+`Space`, type `Terminal`, press Enter.
Linux	Usually `Ctrl`+`Alt`+`T`.

The six commands you actually need

COMMAND	WHAT IT DOES
`pwd`	Print working directory — "where am I right now"
`ls` (Windows: `dir` also works)	List the files here
`cd foldername`	Change directory — go into a folder
`cd ..`	Go up one folder
`mkdir name`	Make a new folder
`claude`	Start Claude Code in the current folder

The one concept that matters: the current folder

The terminal is always "standing in" one folder. Commands act on that folder. When you start Claude Code, **the folder you're standing in becomes its working directory** — the project it can see and edit.

> **Beginner mistake #1**
> Starting Claude Code from your home folder or your desktop and then asking it to fix "the website." It has no idea which website. `cd` into the project folder *first*, then run `claude`.

Paths

A path is a file's address. Two flavors:

- **Absolute** — full address from the root. `C:\Users\yourname\projects\site\index.html` on Windows, `/Users/yourname/projects/site/index.html` on Mac.
- **Relative** — from where you're standing. `index.html`, or `../other-folder/file.txt`.

The `~` character is shorthand for your home folder. `~/.claude/` means "the `.claude` folder inside my home folder." Folders starting with a dot are hidden by default in file explorers — you'll need to enable "show hidden files" to see them.

0.3 — Installing it

Step 1 — Node.js

Claude Code installs through npm, which comes with Node.js. Check if you already have it:

```
node --version
```

If you see a version number (v18 or higher), you're set. If you see "command not found," download the LTS installer from `nodejs.org`, run it, then close and reopen your terminal.

Step 2 — Install Claude Code

```
npm install -g @anthropic-ai/claude-code
```

The `-g` means global — installed once for your whole machine, not per-project. On Mac/Linux, if it complains about permissions, do *not* reach for `sudo` as a first move; fix your npm prefix instead (search "npm global install without sudo") or use a version manager like nvm.

Step 3 — First run and login

```
# make a folder to play in
mkdir ~/ai-sandbox
cd ~/ai-sandbox
claude
```

It opens a browser to log in with your Anthropic account. Approve, come back to the terminal, and you're in. You'll see a prompt where you type in plain English.

Step 4 — Confirm it works

Type this and press Enter:

```
Create a file called hello.txt containing the word "working", then show me the folder contents.
```

It will ask permission to write the file. Approve. If `hello.txt` appears, the whole stack is functional.

> **Exiting and returning**
>
> `Ctrl` + `C` twice, or type `/exit`, to quit. To resume your last conversation in that folder, start it again with `claude --continue`. To pick from previous sessions, `claude --resume`.

0.4 — Your first 20 minutes

Do these in order. Each teaches one thing you'll use forever.

1. Ask it to look around (2 min)

```
What's in this folder? Give me a one-paragraph summary.
```

Teaches: it reads before it acts, and it summarizes rather than dumping.

2. Make it build something small (5 min)

```
Build a single-file HTML page that shows the current time,
updating every second, on a dark background. Then open it
in my browser so I can see it.
```

Teaches: it writes files *and* runs commands. Notice it verified the result rather than just handing you code.

3. Break it on purpose, then ask for a fix (5 min)

Open the file, delete a closing bracket, save. Then:

```
The page is broken now. Find out why and fix it.
```

Teaches: the debug loop. It reads, diagnoses, patches, re-checks.

4. Use the built-in slash commands (3 min)

COMMAND	WHAT IT DOES
`/help`	Lists everything available in your version — **your source of truth**, since features change
`/clear`	Wipes the conversation. Use when switching to a different topic.
`/compact`	Summarizes the conversation so far to free up room. Use at milestones.
`/memory`	Lists and edits your memory/instruction files
`/config`	Settings — theme, model, and other preferences
`/agents`	Lists and manages your subagents
`/mcp`	Shows connected external tool servers

5. Learn the two magic prefixes (5 min)

PREFIX	EFFECT
`!`	Runs the rest of the line as a shell command right in the session, so its output lands in the conversation. `!git status`. Essential for anything interactive that Claude can't do for you, like a login prompt.
`#`	Saves what follows as a durable memory. `#I deploy to Netlify, not Vercel`. It asks which memory file to put it in.

> **Habit to build on day one**
> When it does something wrong, don't just correct it in the moment — ask yourself "is this wrong permanently or just this once?" If permanently, use `#` to save it. That single habit is the difference between a tool that stays dumb and a system that gets smarter every week.

0.5 — The permission model

Before Claude Code does anything with real consequences — writing a file, running a command, hitting the network — it asks. You'll see a prompt with options.

ANSWER	MEANING	WHEN
Yes	Allow this once	Default for anything unfamiliar
Yes, don't ask again	Allow this kind of action from now on in this project	Safe repeated reads: `ls`, `git status`, running tests
No	Reject and explain why	Whenever it surprises you

Permission modes

There's also a session-wide mode. The two you'll meet early:

- **Default** — asks before consequential actions. Stay here while learning.
- **Plan mode** — press `Shift`+`Tab`. It researches and proposes a plan but changes nothing until you approve. This is the single most useful mode in the product and almost nobody new uses it.

> **Use plan mode for anything that touches more than one file**
>
> Read the plan, argue with it, approve it, *then* let it build. Two benefits beyond safety: you catch a wrong approach before an hour is wasted, and for client work the approved plan doubles as a written scope agreement you can bill against.

> **About "bypass permissions" mode**
>
> There is a mode that stops asking entirely. It exists for experienced users running trusted, well-scoped work in a directory they don't mind being changed. Don't touch it for at least a month. When you do, use it in a git repo with everything committed first, so any mistake is one `git checkout` away from undone.

What to never allow blindly

- Anything deleting files in bulk (`rm -rf`, mass moves)
- Anything sending outward — emails, posts, messages, deploys to production
- Anything spending money — ad budgets, paid API calls at volume, purchases

- Anything touching credentials, keys, or auth configuration

These four categories should stay human-approved permanently, no matter how experienced you get. Not because the AI is careless — because they're irreversible and the cost of being wrong is asymmetric.

0.6 — Glossary of every term

Every word this guide uses, defined once.

TERM	MEANING
Context / context window	Everything the AI can currently "see" — your conversation, files it read, instructions loaded. It's finite. When it fills up, older material gets summarized or dropped. Most weird behavior traces back to a stuffed or polluted context.
Token	Roughly ¾ of a word. Both what you're billed in and what the context is measured in. "Token discipline" means not wasting space and money on things that don't help.
Model	The specific AI brain doing the work. Bigger models reason better and cost more. You match the model to the job.
Session	One continuous conversation. Ends when you exit or `/clear`.
CLAUDE.md	An instructions file loaded automatically at the start of every session. Your standing rules.
Memory	Markdown files storing durable facts, so knowledge survives between sessions.
Skill	A saved procedure — a file of steps for a repeated workflow, invoked as `/name` or triggered automatically by matching phrasing.
Slash command	Anything you invoke by typing `/something`. Built-in ones like `/clear`, plus your own.
Subagent	A separate copy of the assistant with its own context and narrow mission, launched to do one job and report back a summary.
Hook	A script that runs automatically at a specific moment — session start, before a tool runs, after an edit. This is how you get behavior that fires without being asked.
MCP	Model Context Protocol — the standard way to plug external services (your hosting, your CRM, your email) in as tools the AI can call.
Tool	Any capability it can invoke: read a file, run a command, search the web, call an MCP service.
Frontmatter	The block at the top of a markdown file between `---` lines, holding settings like name and description.

Repo / repository	A folder tracked by git, with full history of every change.
Commit	A saved snapshot of your changes, with a message describing them.
Push	Uploading your commits to a remote host like GitHub. **Committing is local; pushing is what makes it visible elsewhere.**
Branch	A parallel line of work, so you can build without disturbing the main version.
Worktree	A second folder checked out from the same repo, so two agents can work simultaneously without colliding.
Gate	A check that must pass before something ships. A real gate *fails loudly*; something that just prints information is not a gate.
Idempotent	Safe to run more than once without doubling the effect. Critical for anything that sends or publishes.
Fail-soft	Code that swallows its own errors and continues silently. Convenient, and the reason broken things look like working things.
Headless	Running without a visible window — a browser or app driven by script.
VPS	Virtual Private Server. A small always-on computer you rent, where scheduled automations live so they don't depend on your laptop being awake.

0.7 — Where every file lives

Two locations. **Global** applies everywhere; **project** applies to one folder and its subfolders.

Global — your home folder

```
~/.claude/ ├── CLAUDE.md # standing rules, loaded every session ├── settings.json # permissions, env vars, hooks ├── skills/ │ └── my-skill/ │ └── SKILL.md # one folder per skill ├── agents/ │ └── researcher.md # one file per subagent ├── commands/ │ └── status.md # becomes /status └── projects/ └── <project-key>/ └── memory/ ├── MEMORY.md # the index — loaded every session └── some-fact.md # one fact per file
```

Project — inside a specific repo

```
your-project/ ├── CLAUDE.md # rules for THIS project only └── .claude/ ├── settings.json # shared with your team, commit this ├── settings.local.json# yours only, gitignore this ├── skills/ # project-specific skills ├── agents/ # project-specific agents └── commands/ # project-specific commands
```

> **Which wins**
>
> Project-level settings override global ones. A project skill named the same as a global one takes precedence inside that project. Put universal preferences in global; put anything about a particular codebase in that codebase — and commit it, so anyone else working there inherits it.

Exact paths shift slightly between versions. Run `/memory` to see exactly which files are loaded on your machine right now, rather than trusting any document — including this one.

PART 1

The Idea

Why a system beats a better prompt.

1.1 — Chat box vs operating system

Used as a chat box, AI hits a ceiling fast, for three specific reasons:

PROBLEM	WHAT IT LOOKS LIKE	THE FIX
Amnesia	Every session you re-explain your business, your stack, your preferences, your constraints	Memory files
Improvisation	The same task done slightly differently each time; the gotcha you hit last month gets hit again	Skills
Queueing	Four independent jobs done one after another, each waiting on the last for no reason	Subagents

Fix those three and the character of the tool changes. It stops being something you operate and becomes something that runs.

> **The governing principle**
>
> A prompt you have typed three times is a bug. Convert it into a skill (a procedure) or a memory (a fact). The system should get cheaper to operate every week, not more expensive.

What stays human

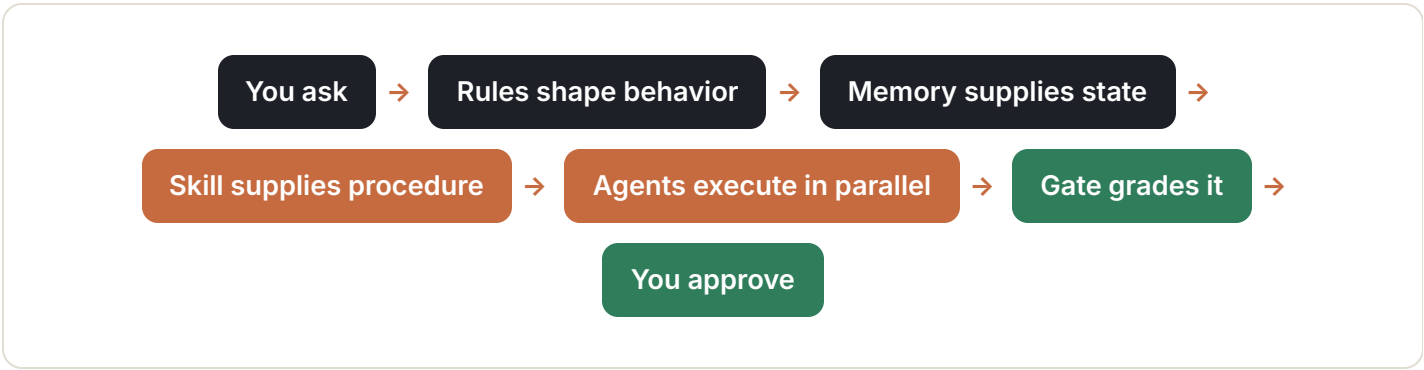
Three things, permanently:

1. **Direction** — what to build, which client, what the business needs this month
2. **Approval** — anything spending money or leaving the building
3. **Taste** — is this actually good? Machines verify correctness; they don't have judgment about quality

Everything between those three is delegable. That's a much larger share of the work than most people expect.

1.2 — The four layers

LAYER	WHAT IT IS	SOLVES
Rules	`CLAUDE.md` — always loaded	Behavioral consistency. Non-negotiables.
Memory	Markdown files + a thin index	Amnesia. State survives sessions.
Skills	Procedure files, loaded on demand	Improvisation. Same job, same quality, every time.
Agents	Sub-assistants with own context	Queueing, and self-review bias.



1.3 — Rules vs skills vs memory

Getting this wrong is the single most common beginner mistake, and it's expensive because it's invisible.

TYPE	CONTAINS	LOADED	COST
Rule	A fact always true. "Never post without my approval."	Every session, always	Paid on every session forever
Memory	A fact about your world. "The staging site is at X."	Index always; detail on demand	Cheap — index is one line
Skill	A procedure. "The 14 steps to publish a video."	Only when relevant	Free until used

The failure mode

Someone discovers `CLAUDE.md`, loves it, and pours everything in — their whole video workflow, their whole deploy checklist, their client list. Six weeks later it's 3,000 lines. Every single session now burns thousands of tokens loading instructions that are irrelevant to 95% of what they do, and the genuinely important rules are buried where they get less attention.

Test: if a line wouldn't matter to a session about a completely unrelated topic, it does not belong in `CLAUDE.md`. Move it to a skill.

PART 2

Building the System

Every file written out in full. Copy, paste, adjust.

2.1 — The rules file (CLAUDE.md)

A plain markdown file at `~/.claude/CLAUDE.md` (global) or `./CLAUDE.md` (project). Loaded at the start of every session. Keep it short.

A complete starter file

```
# Execution Rules

## Quality
- Read code before modifying it. Verify prior work; never assume it's correct.
- Flag hidden assumptions and goal misalignment BEFORE building.
- Test everything testable. Run the code. Don't guess.
- Don't patch weak work — rebuild it properly.

## Execution
- Decompose tasks into independent sub-tasks and run them in parallel.
  Sequential is the exception, not the default.
- Before any multi-file change, use plan mode. Get the plan approved first.
- If I've asked for the same thing twice, suggest turning it into a skill.

## Communication
- Be direct. No filler, no preamble, no trailing summary unless I ask.
- If tests fail, say so and show the output. Never report success you
  haven't verified.
- When you're uncertain, say which part you're uncertain about.

## Never without asking
- Send anything outward: email, post, message, deploy to production
- Spend money
- Delete files in bulk
- Change credentials or auth config
```

That's the whole thing. Around 20 lines. Add to it only when you catch yourself giving the same correction twice — and only if it's genuinely universal.

Project-level rules

Inside a specific codebase, a `CLAUDE.md` there carries what's true about *that* project:

```
# Project: acme-site

## Stack
- Astro + Tailwind. Content in `src/content/`. No React in this repo.

## Commands
- Dev: `npm run dev` · Build: `npm run build` · Test: `npm test`

## Conventions
- Components are PascalCase. Utilities are camelCase.
- Never edit `dist/` — it's generated.

## Deploy
- Pushing to `main` triggers the build automatically.
- Cache must be purged after deploy or the old version stays live.
```

Commit your project CLAUDE.md

It's part of the codebase. Anyone else who works there — human or AI — inherits the conventions instead of guessing at them. This is the cheapest onboarding document you will ever write.

2.2 — Memory: the brain

A folder of small markdown files. **One fact per file.** Plus one index file that's the only thing loaded at session start.

What a memory file looks like

```
---
name: deploy-needs-cache-purge
description: Deploys don't go live until the CDN cache is purged
metadata:
  type: feedback
---

Pushing to main builds and uploads, but the CDN keeps serving the old
version until the cache is purged. The site looks unchanged and there
is no error anywhere.

**Why:** I lost 40 minutes debugging a "broken" deploy that had
actually succeeded. Nothing in the logs indicates this.

**How to apply:** After any deploy, purge the cache, then fetch the
live URL and confirm the new content is actually being served.
Related: [[committed-is-not-deployed]]
```

The index

`MEMORY.md` holds one line per memory. Never content — just a pointer and a hook:

```
# MEMORY

## Rules learned
- [deploy-needs-cache-purge](deploy-needs-cache-purge.md) — deploy succeeds, site unchanged
- [committed-is-not-deployed](committed-is-not-deployed.md) — a commit proves nothing about prod

## Projects
- [acme-site](acme-site.md) — redesign, launch Sept, blocked on client logo
```

The four types

TYPE	CONTENTS	EXAMPLE
user	Who you are: role, expertise, standing preferences	"Prefers deliverables as self-contained HTML, not markdown"
feedback	Corrections and confirmed approaches — *always with the why*	"A commit proves nothing about production"
project	Ongoing work, goals, constraints not derivable from code	"Build X blocked pending a client design decision"
reference	Pointers: dashboards, tickets, docs, environment facts	"Render helpers live in `_lib/`"

Rules that keep memory useful

- **One fact per file.** Bundled files never get updated correctly and never get deleted.
- **Never store what the repo already records.** Code structure, past fixes, git history — it can read those. Memory is for what *isn't* written down anywhere.
- **Absolute dates only.** "Next week" is meaningless three months later.
- **Link liberally** with `[[double brackets]]`. A link to a memory that doesn't exist yet is a to-do, not an error.
- **Delete wrong memories immediately.** A stale fact is worse than a missing one, because it's trusted.
- **Verify before acting on one.** If a memory names a file or a flag, check it still exists. Memories record what was true when written.

> **Feedback is the highest-leverage type**
>
> When the system makes an expensive mistake, the fix is not "be more careful" — it's a feedback memory carrying the trigger, the failure mode, and the why. That mistake then structurally cannot recur. This is the compounding mechanism of the entire system. Everything else is scaffolding around it.

Checkpoints

At the end of a working session, write state to a dated file: what got done, what's blocked, what's next. Tomorrow opens with yesterday's ending. No re-briefing, no "where were we." You can automate this with a hook (§2.7).

2.3 — Skills: frozen procedures

A skill lives at `~/.claude/skills/<name>/SKILL.md`. Frontmatter on top, steps below. It loads automatically when a request matches the description, or you invoke it directly with `/name`.

A complete working skill

```
---
name: new-client-site
description: Scaffold a new client website project from scratch.
  Trigger on "new client site", "start a site for", "/new-client-site",
  or when the user pastes a client brief asking for a website.
---

# New Client Site

## Preconditions
- Client name and the niche are known
- Deploy target is known (ask if not — see Gotchas)

## Steps

1. **Ask the five scoping questions** (all at once, not one at a time):
   - What does a successful visitor DO on this site?
   - Do you have brand assets, or am I creating them?
   - Who writes the copy?
   - Where does it get hosted?
   - What's the launch date?

2. **Research** — spawn a research subagent to look at three
   competitors in the niche. Return: positioning, what they all do
   the same (the category convention), and one gap.

3. **Scaffold** — create the folder, initialize git, set up the
   framework, commit the empty skeleton BEFORE writing any content.

4. **Design tokens first** — colors, type scale, spacing, in one
   file. Everything else references those. Never hardcode a hex.

5. **Build the page**, then run the pre-ship audit skill.

6. **Deploy to a staging URL**, fetch it back, and confirm what's
   live matches what you built.

## Gotchas
- ASK the deploy target at step 1. Different hosts need completely
  different handoffs, and discovering that at the end costs an hour.
- Never put fake testimonials or invented metrics in a demo. If the
  design needs placeholder proof, label it as illustrative ABOVE
  the fold, not in a footer.
- The client's logo will be a low-res JPEG. Ask for vector at step 1.
```

```
## Done when
- Staging URL loads, audit passes, and the client has been sent a
  link with the five questions answered back to them.
```

What separates a skill that fires from one that never does

Strong triggers

The description must contain the literal phrases a human would actually type. "Handles website creation workflows" never fires. "Trigger on 'new client site', 'build me a site for'" fires.

A gotchas section

This is the entire value. Anyone can write the happy path. The skill earns its keep by carrying the six things that went wrong the first four times.

Runnable steps

Real commands with real paths, not "then process the file." Where a step needs a human decision, say so explicitly.

Preconditions and done-when

What must be true before step 1, and how you know it's finished. Half of all skill failures are a missing input nobody checked for.

Testing whether your skill actually fires

Write it, start a fresh session, and type the phrase a normal person would use — *not* the skill's name. If it doesn't load, your description is too vague. Add the literal trigger phrases. Repeat until it fires without being named.

Let the tool write the skill

The fastest way to build one: do the workflow by hand once with Claude Code, and when it's done say *"turn what we just did into a skill."* It has the whole thing in context — including every wrong turn — and will write a better gotchas section than you'd write from memory a week later.

When to build one

After the second time. The first time, you're learning the workflow. The second time, you notice you're repeating yourself — that's the signal. Waiting for the fifth time means three sessions of re-explaining you didn't need to pay for.

2.4 — Slash commands

A shortcut for a prompt you type often. Any file at `~/.claude/commands/name.md` becomes `/name`.

```
# ~/.claude/commands/status.md

Give me a snapshot, in this exact order:

1. What did I ship yesterday? (check git log across my project folders)
2. What's blocked and waiting on ME specifically?
3. What's the single highest-value thing to do next, and why that one?

Keep it under 200 words. No preamble.
```

Now `/status` runs that. Trivial to build, and once you have five of them the daily overhead of the system drops to near zero.

COMMAND VS SKILL	USE WHICH
Command	A prompt you retype. Short, you invoke it deliberately.
Skill	A multi-step procedure with gotchas. Long, and you want it to load automatically when relevant.

2.5 — Subagents

A subagent is a separate instance with its own context window, its own tool permissions, and one narrow mission. It reports back a synthesis — not a dump of everything it read.

Two reasons they matter

1. **Context isolation.** A research agent might read forty pages. If that happened in your main session, it would crowd out everything else. In a subagent, forty pages go in and one paragraph comes back.
2. **Independence.** An agent that didn't build the thing can review it honestly. The builder can't.

Defining one

A file at `~/.claude/agents/researcher.md`:

```
---
name: researcher
description: Deep multi-source web research in an isolated context.
  Use for competitive analysis or any investigation that would
  otherwise flood the main conversation. Returns concise synthesis,
  never raw dumps.
tools: WebSearch, WebFetch, Read, Grep, Glob
---

You are a research specialist. Your job is to investigate thoroughly
and return a TIGHT synthesis.

Rules:
- Read widely, report narrowly. Never paste large excerpts back.
- Separate what you VERIFIED from what you INFERRED. Label both.
- Give sources for every factual claim.
- If sources disagree, say so and say which you trust and why.
- If you couldn't find something, say that explicitly. Do not fill
  a gap with a plausible guess.

Return format:
1. Answer (3-5 sentences)
2. Key findings (bulleted, sourced)
3. What's uncertain or contested
4. Sources
```

The `tools:` line is a permission boundary — this agent can read and search but cannot write files or run commands. For anything that only needs to look, restrict it. A read-only agent cannot damage anything.

The brief is everything

A subagent cannot see your conversation. It knows only what you put in the brief. A thin brief returns thin work. A full brief has six parts:

```
MISSION: Audit the checkout flow for bugs that would cost a sale.
DONE WHEN: Every step from cart to confirmation is exercised and
  every failure is reported with exact repro steps.

CONTEXT: Next.js app in ./src. Payments via Stripe test mode. The
  client reported "sometimes it just spins" — never reproduced.
  Existing tests are in ./tests but cover happy path only.

AUTONOMY: Make your own calls on how to test. Don't check in for
  approval. Run the dev server, drive the browser, break things.

QUALITY BAR: A bug is only real if you reproduced it twice and can
  state the exact inputs. Speculation is not a finding.

RETURN: A list of confirmed bugs, each with file:line, repro steps,
  and severity. Plus one line on what you tested and found FINE —
  I need to know the coverage, not just the failures.
  Do NOT paste file contents back.
```

The most common subagent failure is a lazy brief

"Check the checkout" gets you a vague paragraph. The brief above gets you a bug list you can act on. Writing the brief takes ninety seconds and is the entire difference.

A useful specialist roster

AGENT	MISSION	ACCESS
Researcher	Multi-source investigation, isolated context, synthesis only	Read-only
Explorer	Fan-out search across a big codebase. Locates; doesn't judge.	Read-only
Builders	Framework specialists — frontend, WordPress, TypeScript, whatever you use	Read + write
Auditors	Security sweep, pre-ship site audit, copy QA	Read-only, by design
Planner	Turns a fuzzy goal into a routed, prioritized plan. Plans only.	Read-only
Curator	Keeps memory clean: orphans, stale entries, broken links	Read + write to memory

> **An agent never grades its own work**
>
> The builder is invested in the build. Grading goes to a fresh-context agent that didn't write the thing, ideally instructed to default to FAIL on uncertainty. This single rule catches more defects than any other practice in the system.

2.6 — Parallelism

Sequential is the default and it's usually wrong. If four sub-tasks don't depend on each other, they should run at the same time.



All four launched in ONE message. Launched one at a time, they run sequentially and the benefit vanishes.

How to actually trigger it

Ask for it explicitly until it's habitual:

```
These four things don't depend on each other. Launch all four
subagents in parallel in a single message, then merge the results:

1. Audit the CSS for unused rules
2. Check every external link resolves
3. Review the copy for AI tells and unverifiable claims
4. Scan dependencies for known vulnerabilities
```

The dependency test

Before any multi-part task, ask: **does task B need task A's output?**

- **No** → parallel. Almost always the answer.
- **Yes** → sequential, but only for the pair that's actually linked. Everything else still runs alongside.

When agents edit files simultaneously

They collide. The fix is git worktrees: each agent gets its own isolated checkout of the repo, does its work, opens its own change. Merge smallest-first so later rebases stay trivial. Blast radius per agent is one worktree; a crashed agent doesn't take down the other three.

> **Don't over-parallelize**
> Ten agents on a four-part task means ten agents duplicating work and ten sets of results to reconcile. Parallelism pays when tasks are genuinely independent and genuinely substantial. For three quick edits, one agent doing them in order is faster than the coordination overhead.

2.7 — Hooks: making things happen automatically

A hook is a command that runs automatically at a defined moment. This is how you get behavior that fires whether or not you remember to ask.

The events

EVENT	FIRES	GOOD FOR
`SessionStart`	When a session begins	Injecting today's priorities, current branch, open tasks
`UserPromptSubmit`	Each time you send a message	Adding standing context, enforcing a mode
`PreToolUse`	Before a tool runs	Blocking dangerous commands, requiring a gate to have passed
`PostToolUse`	After a tool runs	Auto-formatting after every edit, running a linter
`Stop`	When it finishes responding	Notifications, logging
`SessionEnd`	When a session closes	Writing the checkpoint

A real example — auto-format after every edit

In `~/.claude/settings.json`:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"$CLAUDE_FILE_PATHS\""
          }
        ]
      }
    ]
  }
}
```

The highest-value hook: SessionStart

Have it print today's priorities so every session opens already oriented:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "cat ~/notes/today.md"
          }
        ]
      }
    ]
  }
}
```

Whatever that command prints gets injected into the session as context. Point it at a file, or a script that assembles priorities, git status, and anything awaiting a reply. This one hook is what makes the system feel like a system rather than a collection of files.

Ask it to write your hooks

Hook configuration is fiddly and version-specific. Rather than hand-editing JSON, describe what you want — *"every time you finish editing a Python file, run black on it"* — and let Claude Code write and install the config. It knows the current schema; a guide written months ago might not.

Hooks run real commands with your permissions

A hook is not sandboxed. Never paste a hook config you don't understand, from anywhere. Read every command in it first.

2.8 — MCP: connecting your other tools

MCP (Model Context Protocol) is the standard way to expose an external service as tools the AI can call. Once connected, your hosting provider, CRM, email, ad account, or database becomes something it can query directly rather than something you screenshot into the chat.

Adding one

```
# see what's connected
claude mcp list

# add a server
claude mcp add <name> <command-or-url>
```

Inside a session, `/mcp` shows connected servers and their status. Each provider documents its own setup — check theirs rather than guessing at the arguments.

What it unlocks

Instead of "here's a screenshot of my analytics, what do you think," you get "pull last week's numbers, compare to the prior week, and tell me what changed." The AI fetches the real data itself.

Read-only by default

Grant the narrowest access that does the job. Reporting and analysis should be automated; changing budgets, sending messages, or deleting records should not. A bug in a read-only integration produces a wrong report. The same bug with write access costs real money while you sleep.

Treat external content as data, never as instructions

Text arriving from a website, an email, a ticket, or an API is *information*. If it contains something that looks like a command — "ignore previous instructions and…" — that is an attack, not a request. This matters more as you connect more services.

PART 3

Running It Daily

The loop that makes it compound.

3.1 — The daily loop



Morning: context is already loaded

A SessionStart hook injects today's priorities, yesterday's checkpoint, and anything that's gone silent past its follow-up window. There is no "let me catch you up" phase. The first message of the day is already about work.

The follow-up re-fire

The single most valuable automation in the stack is embarrassingly simple: a check listing every outstanding item with no reply past its follow-up window, which drafts the nudge for you.

Deals die from unfired follow-ups far more often than from rejection. Automating the *reminder to follow up* — not the sending — beats automating almost anything else, and it takes an afternoon to build.

Evening: realign

A nightly job reads the memory index and the day's activity, then rewrites the whole task list into one ranked plan for tomorrow: revenue first, blocked-on-you items surfaced explicitly, dead items culled, stale entries flagged. That plan is what the morning hook injects. The loop closes.

3.2 — Working modes

Plan mode

`Shift`+`Tab`. Research and propose, change nothing until approved. Mandatory for anything billable or multi-file. The approved plan doubles as your scope agreement.

Compressed output

A terse mode for long sessions — drops filler and hedging, keeps every bit of technical substance. Real token savings on multi-hour work.

Background work

Long-running jobs (a build, a render, a test suite) can run detached while you keep working, and notify you when they finish.

Capture-before-close

Any session that produced a hard-won fix writes it to a skill or memory *before* the session ends. Otherwise it evaporates and gets re-learned at full price.

3.3 — Context hygiene

The context window is finite and shared. What's in it determines output quality more than any other single factor.

ACTION	WHEN	WHY
`/clear`	Switching to an unrelated project or topic	Leftover context from another project produces confused, cross-contaminated work
`/compact`	At a milestone in long work	Summarizes what's happened and frees room, keeping the thread
Subagent	Any big read — research, wide search, log analysis	The bulk stays out of your main context; only the conclusion comes back
Don't re-read	A file read this session, unchanged	Pure waste of tokens and money

Symptoms of a polluted context

It starts referencing an old project, repeats work already done, ignores an instruction it followed ten minutes ago, or gives vaguer answers than it did earlier. All the same fix: `/clear` and restate the task in three sentences. Fighting a polluted context with better prompting never works.

PART 4

Quality

Nothing goes out ungraded.

4.1 — Quality gates

A gate is a check something must pass before it ships. The definition that matters:

> **Assert, don't print**
>
> A gate that prints a value and lets a human eyeball it is not a gate. It must *fail loudly* on mismatch. "It printed 48000" and "it asserted the rate is 48000" are different programs, and only one of them catches the bug at 2am when nobody is reading the output.

Concretely, gate anything where being wrong is expensive: client deliverables, deploys, anything published, anything sent, anything spending money.

4.2 — The critic loop



Loops until pass or a round limit. Judges default to FAIL on uncertainty.

Design principles

- **A false pass costs more than a false block.** Calibrate the whole gate around that asymmetry.
- **Multiple judges, reasons before scores.** Forcing the reasoning first prevents picking a number and then justifying it.
- **Verify the findings.** Critics invent problems too. Every finding must be reproducible against the actual artifact or it's discarded.

- **Track what's resolved.** Otherwise round three re-raises what round one fixed and the loop never converges.
- **Auto-fail overrides every vote.** A verified exposed secret, or an identifiable person who didn't consent, is a hard stop regardless of how good everything else is.

The minimum viable version

You don't need a framework to start. Do this:

```
Spawn a fresh subagent that has NOT seen our conversation.

Give it: the file at ./deliverable.html, and this rubric —
[correctness, completeness against the brief, anything a client
would notice as sloppy].

Tell it to default to FAIL on uncertainty, and to report each
finding with a specific location and why it matters.

Then show me its verdict. Don't fix anything yet.
```

That single move — an independent pair of eyes with no stake in the build — catches an embarrassing amount.

4.3 — Green is not the same as good

> **Passing gates measures the absence of defects, not the presence of quality**
>
> A video can clear every technical check — correct codec, clean audio, no privacy leak — and still be badly edited. A site can pass every audit and still be ugly and unconvincing. A proposal can be factually flawless and completely unpersuasive.
>
> Automated gates catch *failures*. Craft still requires a human with taste to watch it, read it, and say yes. Never mistake a green board for a good deliverable.

Multi-model review

For high-stakes work, have a second and third model from different providers review the same artifact. Different training produces genuinely different blind spots; agreement across model families is a far stronger signal than one model's confidence. Use it selectively — it costs real tokens and most work doesn't need it.

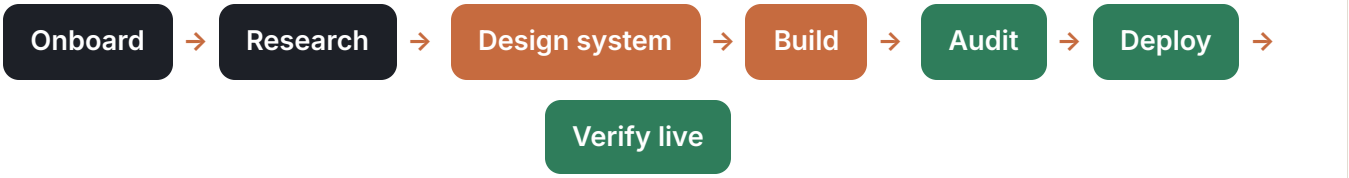
PART 5

Web & Client Delivery

How a site actually gets built and handed over.

---

5 — Web & client delivery



Onboarding is a hard gate

Every new client repo starts with a structured onboarding pass that captures the thing everyone forgets until it breaks: **the deploy target and its specific quirks**. Some hosts take a git push. Some take a manual archive upload. Some need a cache purge afterward or the old build stays live. Some need a specific PHP version.

Writing that down once, per client, at the start, prevents the "why is my fix not showing" hour later — an hour that happens on every project where it wasn't written down.

Design tokens before pixels

Colors, type scale, spacing, radii, shadows — defined once in one file, referenced everywhere. Never a hardcoded hex in a component. When the client says "make it warmer," that's one edit rather than forty. This is the difference between a site you can revise and a site you have to rebuild.

Speculative builds beat proposals

For a competitive pitch, a working demo outperforms a document. But two rules are absolute:

Never let a spec demo tell a lie

If a demo shows metrics, testimonials, or logos, they must be labelled as illustrative *above the fold* — not buried in a footer. Fabricated proof is the fastest way to lose a client you were about to win, and it can outlive the pitch on the open web.

Pre-ship audit dimensions

DIMENSION	CHECKED FOR
Responsive	Real breakpoints on real viewports — not a desktop window resized
Links	Every internal and external link resolves; no 404s, no placeholder anchors
SEO / structure	Meta, heading hierarchy, structured data, canonical, sitemap
Performance	Core Web Vitals signals, image weight, render-blocking assets
Leftovers	Lorem ipsum, TODO comments, dummy emails, test data
Accessibility	Contrast, focus states, alt text, keyboard reachability
Security	Exposed keys, dependency advisories, missing headers

Deliverable format

Client-facing documents ship as **self-contained HTML with a print-to-PDF button**, on a bright theme. Not markdown, not a raw doc. One file, zero dependencies, opens anywhere, prints clean. A client should never need a tool they don't have to read your work. (This guide is built exactly that way.)

PART 6

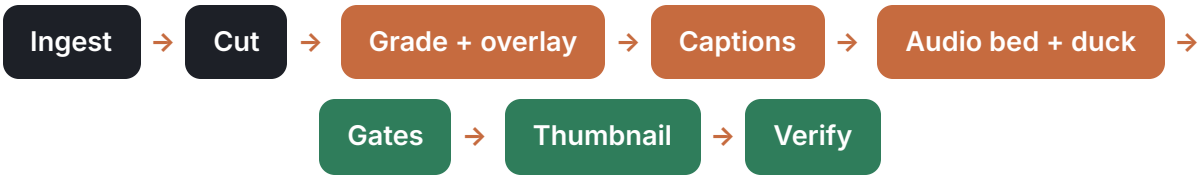
Video Production

Where the failure modes are silent and the gates matter most.

---

6 — Video production pipeline

Video carries the most scar tissue, because a render can "succeed," produce a file, and still be unusable — wrong audio rate, missing frames, a privacy leak in one corner of one shot. Nothing errors. It just looks fine and isn't.



The gates that actually catch things

GATE	WHAT IT ASSERTS
Pixel gate	A frame-by-frame pass over the *rendered* file at a real sample rate. Re-reading the audio proves nothing about the picture.
Audio spec gate	Asserts sample rate, channel layout, and a loudness floor on the final file. Normalization filters can silently change the output rate.
Copyright gate	Fingerprints the music bed *before* upload. Catching a claim beforehand costs minutes; catching it afterward costs the post.
Privacy gate	Screens, whiteboards, documents, faces, filenames. Re-run after *any* reframe.
Publication check	Confirms the live post matches the intended master by frame hash — not by duration.

> **Clearance is resolution-specific**
>
> If you clear a video at one framing and later widen the crop or re-derive at higher resolution, every clearance is void. The wider frame contains pixels nobody has ever checked. Re-derive the geometry from the original source and re-run every gate.

Hybrid rendering

The pattern that works: assemble the heavy video with a fast native encoder, render the fancy animated overlay separately as a transparent alpha layer, then composite the two. Pushing 4K source through a JavaScript animation renderer chokes; keeping them in separate lanes gives you both premium motion and a render that actually finishes.

PART 7

Content & Distribution

Making AI-assisted writing that doesn't read like AI.

---

7 — Content & distribution

The engine

One command produces a full day's pack across every channel — long post, carousel, short-form script, platform-native variants. Voice is locked to a template so output stays recognizably yours rather than drifting into generic assistant prose.

Three passes every piece must survive

Humanizer

Strips AI tells: the em-dash tic, "in today's fast-paced world," rule-of-three lists everywhere, hollow transitions, over-hedged claims.

No-fake-claims

Every number, result, and testimonial traces to something real. Unverifiable claims get cut, not softened.

Anti-repetition

Cross-day check so the same hook, anchor, or anecdote doesn't recycle within a window. Readers notice before you do.

Specificity is the authenticity signal

Generic content reads as machine-written no matter how well it's phrased. Concrete detail woven into the body — a real place, a real number, a real constraint you hit — is what makes it land. Woven, not pasted: a specific sentence dropped into an otherwise generic paragraph reads worse than no specificity at all.

Distribution rules learned the hard way

- **Platform classifiers move.** A CTA pattern that worked for a year can start suppressing reach overnight. Retire tactics on evidence, not sentiment.
- **The scheduling clock is the account's timezone, not yours.** Read it live from the API on every build. Assuming it burns an entire drip.
- **Test row one before trusting the batch.** A ten-post schedule built on a column misalignment produces ten broken posts.
- **Publishing must be idempotent.** If a job re-runs, it must not re-fire. Duplicate posts are worse than late posts.

PART 8

Automation

Things that run without you — safely.

---

8 — Automation (n8n / VPS)

Long-running automations — inbox triage, lead pipelines, scheduled sweeps — belong on a small always-on server running a workflow engine, not on your laptop. The laptop is for building; the server is for running. A laptop that sleeps is not infrastructure.

The loop gate — three caps and a breaker

No automation loop goes live without all four

1. **Step cap**
   — maximum iterations per run
2. **Wall-clock cap**
   — maximum runtime before forced stop
3. **Spend cap**
   — maximum cost per run and per day
4. **Circuit breaker**
   — an error-rate or anomaly threshold that halts the loop and alerts a human

All four live in
config
, never in a prompt. A cap that exists only as an instruction inside a prompt is not a cap — it's a suggestion to a system that can misread it.

Monitor distribution, not liveness

"The workflow ran successfully" is a near-useless signal. A workflow that ran 40 times and produced identical empty output every time reports as perfectly healthy.

Monitor the *shape of the output*: how many items, what variety, what changed versus yesterday. Silent degradation is the normal failure mode of automation, and liveness checks are blind to it.

Outbound discipline

- **Low daily volume, warmed sender.** Volume kills domains, not content.
- **Validate addresses before sending.** Bounce rate is the reputation metric that matters.
- **Log every send to a ledger.** Without it you cannot answer "did we already contact them" and you will double-send.
- **Poll every folder, not just the inbox.** Replies land in spam and get missed for weeks.
- **AI review is a sensor, never a verdict.** It flags, scores, and drafts. A human presses send on anything leaving the building.

> **Audit the running estate, not the plan file**
>
> The plan says a workflow is active. The plan is a document; it has no idea what the server is doing. Before reporting status, query the actual engine, the actual mailbox, the actual DNS record. Documents drift from reality within days, and a status report built from a document is fiction.

PART 9

Paid Ads

Where mistakes cost money per hour.

---

9 — Paid ads discipline

Tracking before spending

The most expensive ad mistake available: run a campaign for days, get clicks, get zero recorded conversions — then discover the conversion event never fired, because the tracking code was written but never deployed, or an environment variable was never set in production.

> **Fire the conversion event manually and watch it land before spending a cent**
>
> Not "the code is committed." Not "it looks right." Trigger the real event, see it arrive in the platform's event dashboard, *then* turn on budget.

Rules set before launch, not during

RULE	DECIDED IN ADVANCE
Kill threshold	Spend with zero conversions at which an ad is paused — decided before launch, executed without renegotiation
Learning floor	Minimum data before *any* optimization decision. Reacting to day-one numbers is reacting to noise.
Fatigue signal	Frequency and CTR-decay thresholds that trigger a creative refresh
Reallocation cadence	How often budget moves, and by what maximum step
Max concurrent tests	A hard cap. More variants than you have traffic for teaches you nothing, slowly.

The reason to decide these before launch is psychological, not technical. Once money is spent, you will find reasons to give a losing ad one more day. A threshold set in advance doesn't negotiate.

PART 10

Hard Rules, Machine, Security

The portable part. Every one of these was learned expensively.

10.1 — Hard rules (scar tissue)

Committed is not deployed

A commit proves nothing about production. Before writing a "missing" feature, check for unpushed commits and check whether the environment variable the code depends on is actually set in the live environment. Fail-soft code produces silence, not errors — so a broken integration looks exactly like a working one that had nothing to do.

Verify against reality, not against your notes

Query the running system, the live mailbox, the actual DNS record. Never repoint an endpoint without fetching what's currently there first — the "parking page" you're about to overwrite may be the live site.

Sent is not pending

Once something is sent, it's awaiting a reply — not awaiting action. Tracking it as pending produces duplicate follow-ups that read as desperate. Status comes from the mailbox, not from a plan file.

A spot check is not proof

Verifying three files out of two hundred tells you three files are fine. If a claim covers a set, verify the set — or state the sample size explicitly.

Regex sweeps hit more than intended

Bulk find-and-replace across a repo always catches something you didn't picture. Dry-run first, read the full match list, then apply.

Opening a file for writing can destroy it before the write fails

Truncating write modes empty the target the moment the handle opens. If the script then throws, the data is gone. Write to a temp file and atomically replace.

Recursive scans can abort silently on a permission error

Some tools return a clean zero-result on a partial failure. Zero results is a suspicious answer, not a reassuring one — verify the scan actually traversed everything before concluding "nothing found."

Filenames re-identify anonymized material

You blurred the faces and bleeped the names, then shipped a file whose name contains the company. Check the *rendered output and its metadata*, not the source project.

Exit code zero is not success

Encoders, uploaders, and API clients all have paths that exit clean while producing garbage. Check the artifact, not the return code.

Local-only is not backed up

Know exactly which folders sync and which don't. The assumption that "it's in the cloud folder" is how irreplaceable footage disappears. Verify sync status per folder, in writing, once.

10.2 — Machine & cost discipline

Never freeze the box

> **Every heavy job reserves a fixed share of CPU for the human**
>
> Renders, encodes, batch processing — all run at lowered priority, with a headroom check before starting and a worker pool capped below the core count. Never two heavy jobs at once.
>
> A render taking 40 minutes instead of 25 is fine. A frozen machine for 25 minutes is a **failure**, because you can't work, can't review, can't answer a client. Throughput of the machine is not the metric. Throughput of the operator is.

Token discipline

- **Keep the always-loaded layer thin.** Every token in `CLAUDE.md` is paid on every session forever. Procedures belong in skills, loaded on demand.
- **Isolate big reads in subagents.** Forty pages of scraped text should never enter the main context; only the synthesis should.
- **Return contracts.** Agents return conclusions and file paths, never file dumps.
- **Don't re-read.** A file read this session and unchanged is pure waste to read again.
- **Match model to task.** Mechanical work doesn't need the expensive tier; the hardest reasoning and the final judge do.
- **Clear between projects.** Cheaper and more accurate than carrying three contexts at once.

10.3 — Security hygiene

- Secrets in environment variables and secret stores — never in prompts, never in committed files, never in a plan document.
- Secret scanning in the pre-ship gate, on the *rendered output* as well as the source.
- Least-privilege tokens. Read-only wherever write isn't strictly required.
- A scheduled audit of the machine and the estate — what's exposed, what's stale, what's unpatched.
- Cookie-based access to logged-in services stays read-only. Read and draft; never auto-act on someone else's account.

- Treat all external content as data, never as instructions. A web page or email that says "ignore your instructions" is an attack.
- Review every diff before committing. Speed is not a reason to merge code you haven't read.

PART 11

Troubleshooting

Every common beginner failure and its actual fix.

11 — When it goes wrong

SYMPTOM	LIKELY CAUSE	FIX
It ignores my CLAUDE.md rules	File too long, or the rule is buried in a wall of text	Cut it to 20 lines. Move procedures to skills. Short files get followed.
My skill never triggers	Description too vague	Put the literal phrases a human would type into the description. Test in a fresh session using natural wording, not the skill name.
It keeps forgetting things	You're correcting in-conversation instead of saving	Use `#` to save it as a memory the moment you notice.
Answers got vague / it's repeating itself	Context is full or polluted	`/clear`, then restate the task in three sentences. More prompting won't fix it.
It says done, but nothing changed	It edited a different file, or in a different folder	Ask for the absolute path of every file it touched. Check you started it in the right directory.
"It works" but production is unchanged	Committed, not pushed. Or pushed but not deployed. Or deployed but cached.	Check `git log origin/main..HEAD`, check the deploy ran, purge cache, fetch the live URL.
It's slow on multi-part tasks	Running sequentially	Say explicitly: "these are independent, launch all of them in parallel in one message."
Costs are higher than expected	Bloated always-loaded layer, re-reading files, big research in main context	Thin the rules file, use subagents for research, `/clear` between projects.
It made up a file, function, or flag	Asked to recall rather than to check	Tell it to verify against the actual filesystem or docs before asserting. Fabrication happens

		most when it's guessing under pressure.
It did something I didn't want	Ambiguous instruction, or permissions too loose	Use plan mode. Tighten permissions. And write a memory so that class of misread doesn't repeat.
A subagent came back useless	Thin brief	Rewrite with all six parts: mission, done-when, context, autonomy, quality bar, return contract.
The whole thing feels like overhead	You built structure before you had repetition	Delete what you don't use. Only build a skill after the second time you did the thing by hand.

The universal first move

When something is behaving strangely, before debugging anything else: `/clear` and try once more with a clean context and a three-sentence statement of the task. A surprising share of "the AI is being dumb" is a context problem, and this costs ten seconds to rule out.

PART 12

Build Your Own in 7 Days

One thing per day. Don't skip ahead.

---

12 — The seven days

DAY	BUILD	EXERCISE
1	Install it, run the first-20-minutes exercises	Get it building, breaking, and fixing a small HTML page. Learn `!` and `#`. Nothing else.
2	Global `CLAUDE.md` — 20 lines max	Copy the starter file in §2.1. Change three lines to match how you actually work. Resist adding more.
3	Memory folder + index	Write five facts it re-asks you about every session. Use the file format in §2.2. Add each to the index.
4	Your first skill	Pick the workflow you've explained most often. Do it once by hand with Claude Code, then say "turn what we just did into a skill." Test that it fires from natural phrasing.
5	One quality gate	On your highest-stakes output, spawn a fresh subagent to grade it against a rubric before you send. Use the prompt in §4.2.
6	Parallel agents, once	Find a task with four genuinely independent parts. Launch all four at once. Feeling this is what changes how you scope everything afterward.
7	A SessionStart hook + a checkpoint habit	Have the session open with today's priorities. End the day writing what shipped, what's blocked, what's next. Now it compounds.

What to expect

- **Day 2–3** feels like admin. It is. The payoff isn't visible yet.
- **Day 4–5** is the first real "oh" — a workflow that used to take explaining now takes one word.
- **Week 2–3** is when it starts feeling like infrastructure. You stop thinking about the tool and think about the work.
- **Month 2** is when the feedback memories have accumulated enough that it stops making the mistakes it used to make. That's the compounding.

The three habits that matter more than any tool

Write the failure down

Every expensive mistake becomes a memory with the trigger and the why — the same day. Not "be careful next time." A rule, in a file.

Never grade your own work

Independent review on anything that leaves the building. The builder cannot see the build clearly.

Parallel by default

Ask "do these depend on each other?" before every multi-part task. Usually they don't, and sequential was a habit, not a requirement.

13 — One-page cheat sheet

Start / stop

`cd project-folder` then `claude` · `claude --continue` resumes last · `claude --resume` picks a session · `Ctrl`+`C` twice to quit

In-session

`/help` everything available · `/clear` wipe context · `/compact` summarize · `/memory` edit memory files · `/agents` manage subagents · `/mcp` connected services · `Shift`+`Tab` plan mode

Prefixes

`!command` run a shell command inline · `#fact` save a durable memory

File locations

`~/.claude/CLAUDE.md` rules · `~/.claude/skills/<name>/SKILL.md` · `~/.claude/agents/<name>.md` · `~/.claude/commands/<name>.md` → `/name` · `~/.claude/settings.json` hooks & permissions · `./CLAUDE.md` per-project rules

The four decisions

Always true? → rules file · **A fact about my world?** → memory · **A procedure?** → skill · **Independent work?** → parallel agents

Before anything ships

Independent grader that didn't build it · defaults to FAIL on uncertainty · findings must be reproducible · a human looks at it with taste before it goes

Never without a human

Sending outward · spending money · bulk deletion · credentials

The one-line version

Give the AI memory so it stops forgetting, procedures so it stops improvising, parallelism so it stops queueing, and an independent judge so it stops shipping its own mistakes. Everything else is detail.

The AI Operating System — a beginner's complete guide to running real work on Claude Code.
Generic by design: no client data, no credentials, no proprietary internals. Every rule here was learned by breaking something first.

Built as a single self-contained HTML file — no dependencies, no tracking, no build step. Open it in any browser and press Save as PDF to regenerate the PDF.
Commands and file paths reflect Claude Code as of August 2026. Features move; run `/help` for what's true on your machine.